锁存器与触发器

锁存器与触发器是时序电路的基本构件

锁存器：不断监测输入，并独立于时钟信号，且在任何时刻都可以改变输出。

触发器：不断监测输入，但只在时钟信号所确定的时刻（非时间段）改变输出。

lilinear@zju.edu.cn

Verilog中阻塞赋值与非阻塞赋值

- （1）在描述组合逻辑的always块中用阻塞赋值，则综合成组合逻辑的电路结构；
- （2）在描述时序逻辑的always块中用非阻塞赋值，则综合成时序逻辑的电路结构。

lilinear@zju.edu.cn

阻塞赋值

- 阻塞赋值操作符用等号(即 =)表示。
- “阻塞”是指在进程语句（initial和always）中，当前的赋值语句阻断了其后的语句
- 后面的语句必须等到当前的赋值语句执行完毕才能执行。
- 阻塞赋值可以看成是一步完成的
- 即：计算等号右边的值并同时赋给左边变量

lilinear@zju.edu.cn

阻塞赋值

- 阻塞赋值操作符用等号(即 =)表示。
- G1 = next_Q;
- G2 = G1;
- 结果是G2= next_Q

lilinear@zju.edu.cn

非阻塞赋值

- 非阻塞赋值操作符用小于等于号 (即 <=)表示。
- “非阻塞”是指在进程语句（initial和always）中，当前的赋值语句不会阻断其后的语句。
- 非阻塞语句可以认为是分为两个步骤进行的：
- ①计算等号右边的表达式的值
- ②在本条赋值语句结束时，将等号右边的值赋给等号左边的变量。

lilinear@zju.edu.cn

非阻塞赋值

- 非阻塞赋值操作符用等号(即 <=)表示。
- G1 <= next_Q;
- G2 <= G1;
- 结果是G2= G1

lilinear@zju.edu.cn

阻塞赋值是按需执行，非阻塞赋值是并行执行。

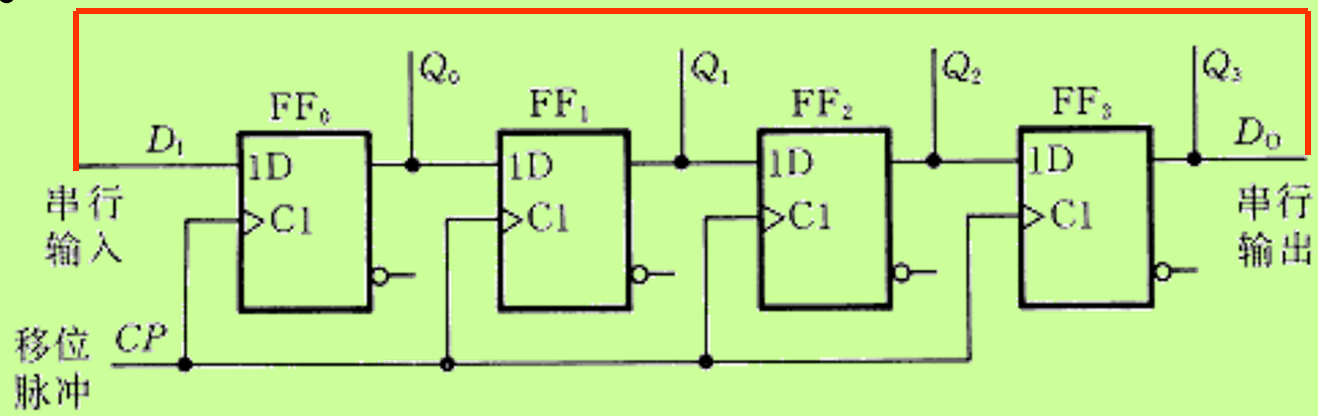
lilinear@zju.edu.cn

移位寄存器型计数器

lilinear@zju.edu.cn

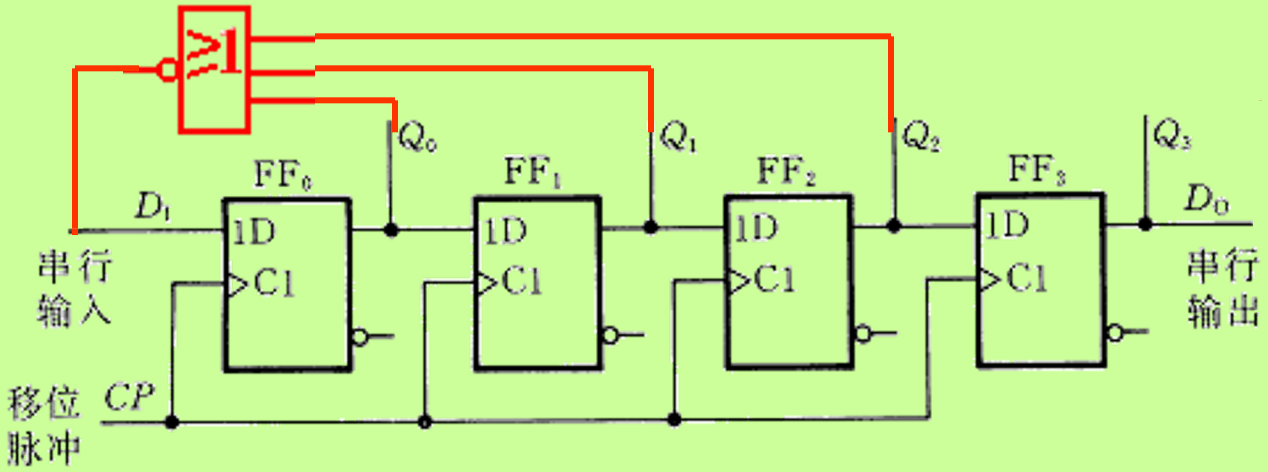
1. 环形计数器

如果把移位寄存器的首尾连接起来，即把串行输出接回到串行输入上，那么在时钟信号的作用下，寄存器里的数据将循环右移。



假定电路的初始状态为$Q_3Q_2Q_1Q_0$ =0001，那么在时钟的不断作用下，电路状态将按 0001→0010 →0100 →1000 →0001次序循环变化。因此可以用不同状态来表示输入时钟的数目。也就是说，实现了一个对时钟脉冲的计数器。

lilinear@zju.edu.cn



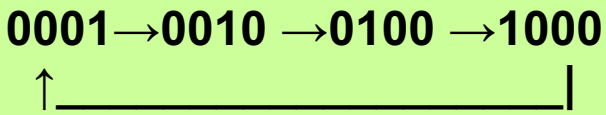
但是这个电路不能自启动。

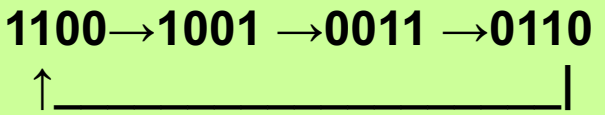
如1100这个无效状态，它不能自动退出。因此必须进行修改。

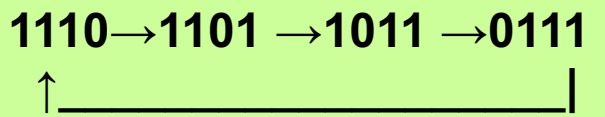
修改后的电路在输出与输入之间增加了反馈电路。如图。

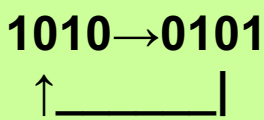
经过这样修改后就构成了能自启动的 4 位环形计数器。

lilinear@zju.edu.cn









0000

1111

lilinear@zju.edu.cn

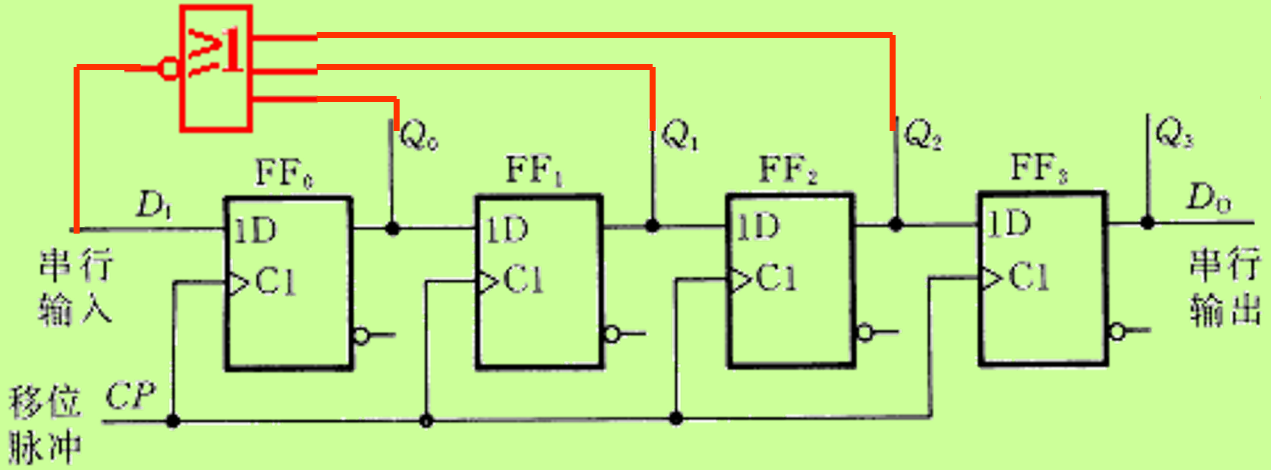
$Q_3Q_2Q_1Q_0$	$Q_3^*Q_2^*Q_1^*Q_0$
0000	0001
0001	0010
0010	0100
0011	0110
0100	1000
0101	1010
0110	1100
0111	1110
1000	0001
1001	0010
1010	0100
1011	0110
1100	1000
1101	1010
1110	1100
1111	1110

Q_0^*

Q_3Q_2 \ Q_1Q_0	00	01	11	10
00	1	0	0	0
01	0	0	0	0
11	0	0	0	0
10	1	0	0	0

$Q_0^* = ?$

lilinear@zju.edu.cn

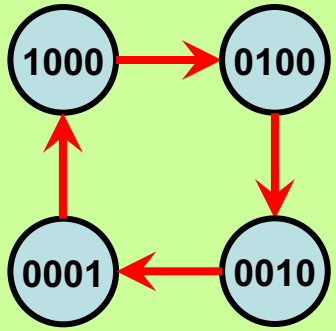


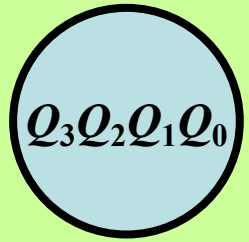
lilinear@zju.edu.cn

移位寄存器构成的移位型计数器

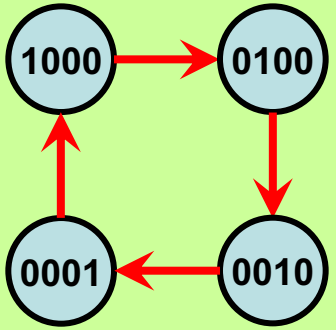
环形计数器的特点：

电路简单，N位移位寄存器可以计N个数，实现模N计数器。状态为1的输出端的序号等于计数脉冲的个数，通常不需要译码电路。





lilinear@zju.edu.cn



环形计数器的优点：

1. 电路结构极其简单。
2. 每个有效状态只含一个 1，可以用触发器输出端的 1 状态表示电路的一个状态，不需要另外加译码器。

环形计数器的突出缺点：

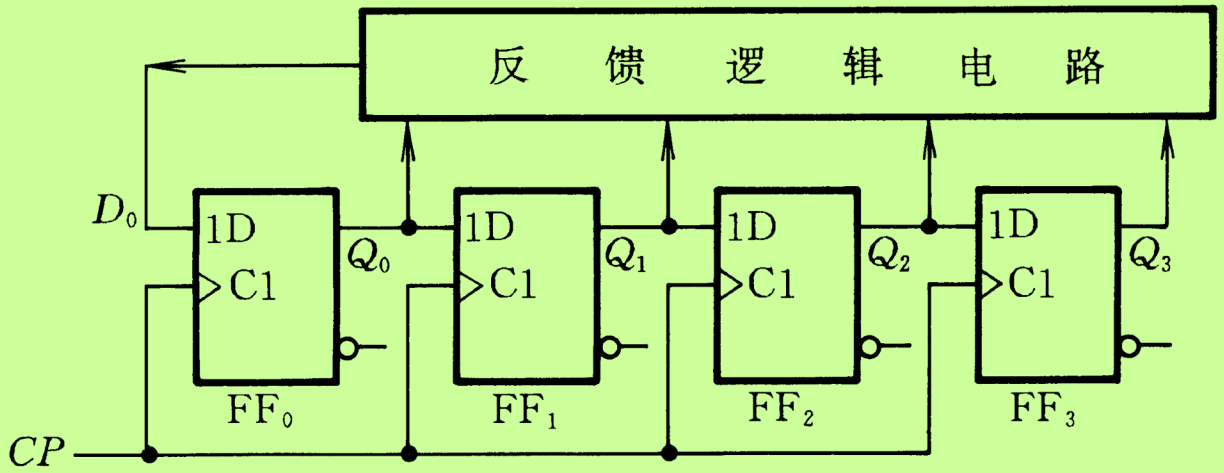
1. 没有充分利用电路的状态， 用 n 位的寄存器组成环形计数器只用了 n 个状态。而电路总共有 2^n个状态

lilinear@zju.edu.cn

2、扭环形计数器

为了提高环形计数器的电路状态利用率，可以改变反馈逻辑电路的结构。

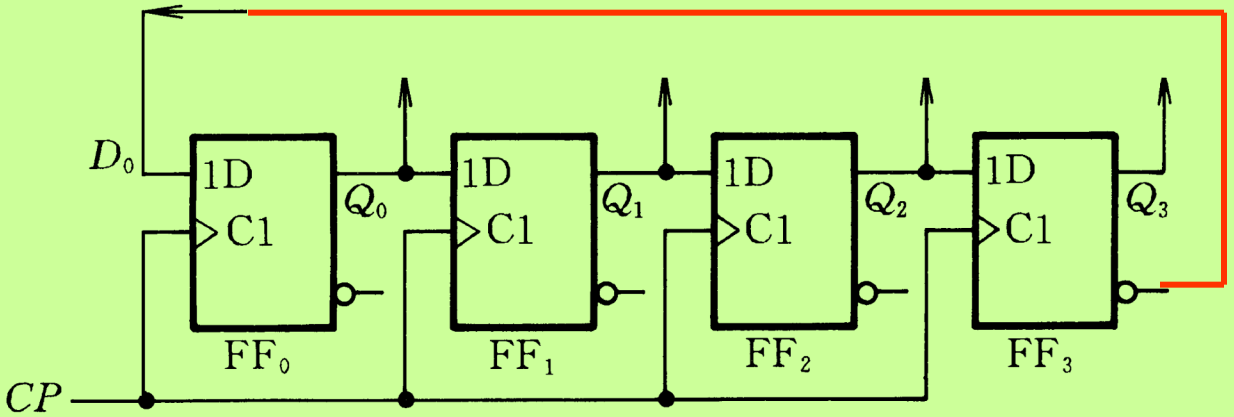
一般的，任何一种移位寄存器型计数器都可以表示为一般的形式：



其中 $D_0 = F(Q_0, Q_1, Q_2, \cdots\cdots Q_{n-1})$

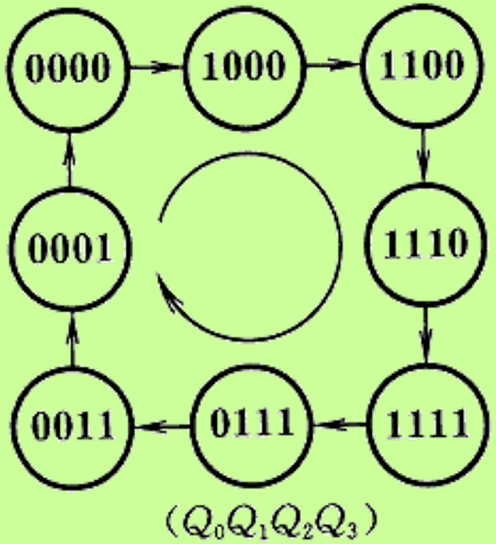
对于环形计数器，$D_0 = Q_{n-1}$，是最简单的一种。

lilinear@zju.edu.cn



如果反馈逻辑函数 $D_0=\overline{Q_{n-1}}$ ，那么电路就变成上图的形式。

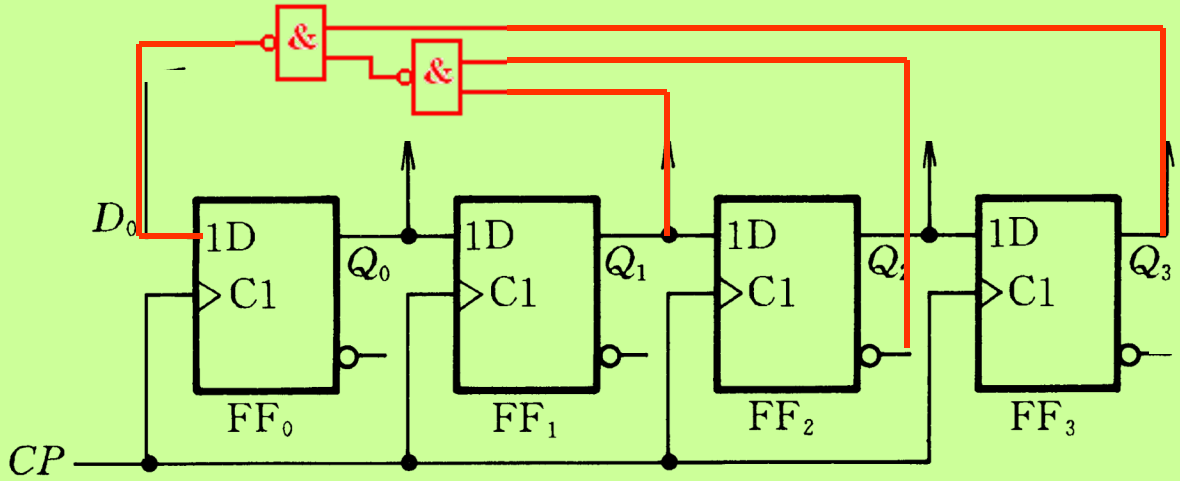
这个电路称为扭环形计数器。它的状态转换图为：



显然扭环形计数器的有效状态为$2n$个，比环形计数器的利用率提高了一倍。

而且每次状态转换时只有一个触发器改变状态，所以译码时不会出现竞争-冒险现象

lilinear@zju.edu.cn



同样这个电路也是不能自启动的，为了实现自启动，需要修改反馈电路。修改后的反馈函数为：

$$D_0 = Q_1\overline{Q_2} + \overline{Q_3}$$

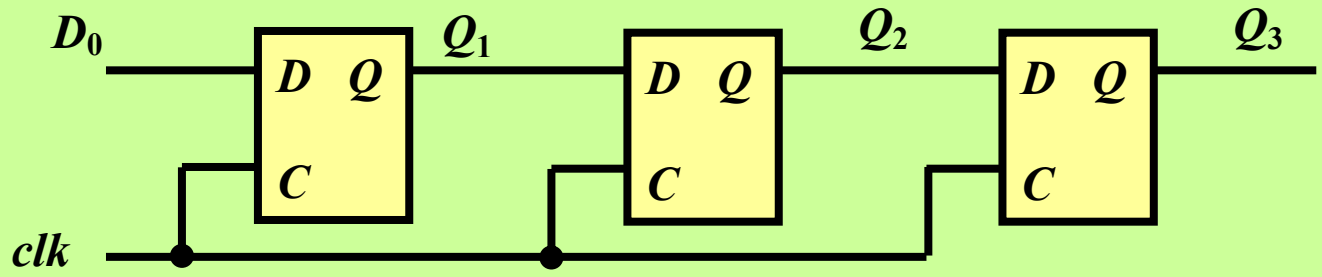
修改后的电路如上图。

lilinear@zju.edu.cn

移位寄存器举例

```
module Right_Shift_reg (Q3, D0, clk);
          output [n:0] Q3;
          input [n:0] D0;
          input clk;
          reg [n:0] Q3, Q2, Q1;
          always @(posedge clk)
          begin
                    Q1 = D0;
                    Q2 = Q1;
                    Q3 = Q2;
          end
endmodule
```

请判断是否正确?



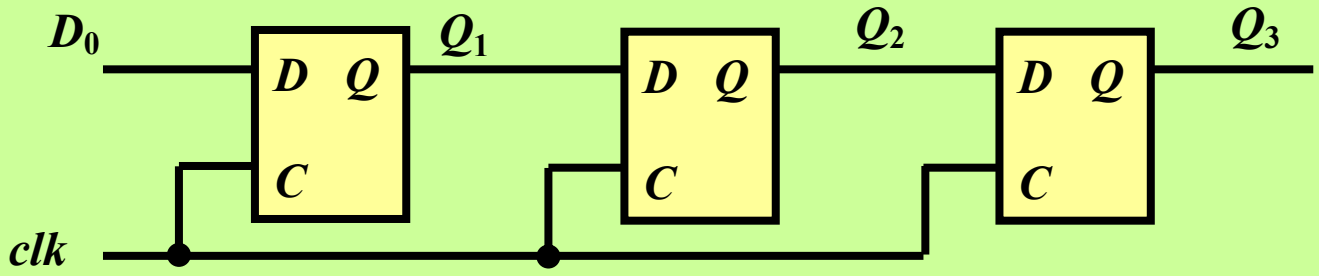
lilinear@zju.edu.cn

移位寄存器举例

```
module Right_Shift_reg (Q3, D0, clk);
          output [n:0] Q3;
          input [n:0] D0;
          input clk;
          reg [n:0] Q3, Q2, Q1;
          always @(posedge clk)
          begin
                    Q1 <= D0;
                    Q2 <= Q1;
                    Q3 <= Q2;
          end
endmodule
```

```
Q3 <= Q2;
Q2 <= Q1;
Q1 <= D0;
```

```
always @(posedge clk) Q3 <= Q2;
always @(posedge clk) Q2 <= Q1;
always @(posedge clk) Q1 <= D0;
```



lilinear@zju.edu.cn

- 在描述组合逻辑的always 块中用阻塞赋值，则综合成组合逻辑的电路结构。
- 在描述时序逻辑的always 块中用非阻塞赋值，则综合成时序逻辑的电路结构

lilinear@zju.edu.cn

一种真正的边沿触发器
维持阻塞触发器

lilinear@zju.edu.cn

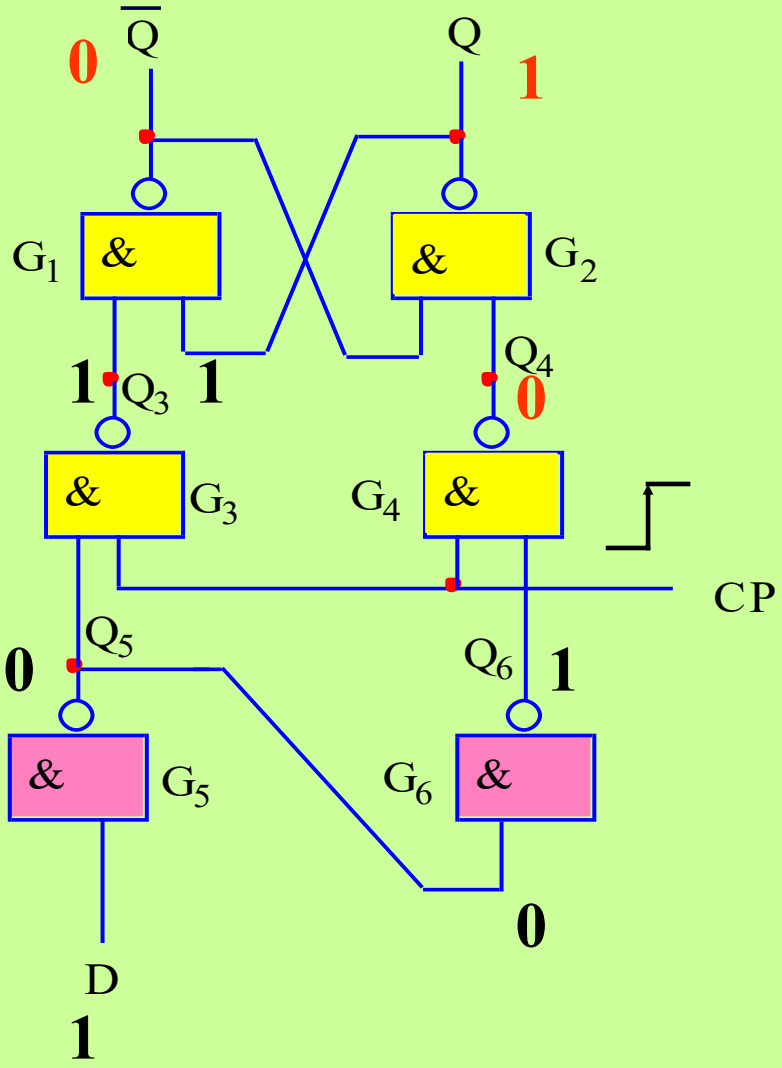
维持—阻塞边沿D触发器的结构及工作原理

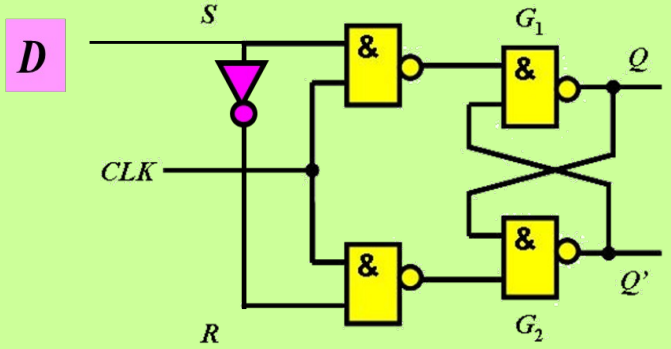
（1）同步D触发器：

设：D=1

该触发器为上升沿触发。

该电路满足D触发器的逻辑功能，但有同步触发器的空翻现象。

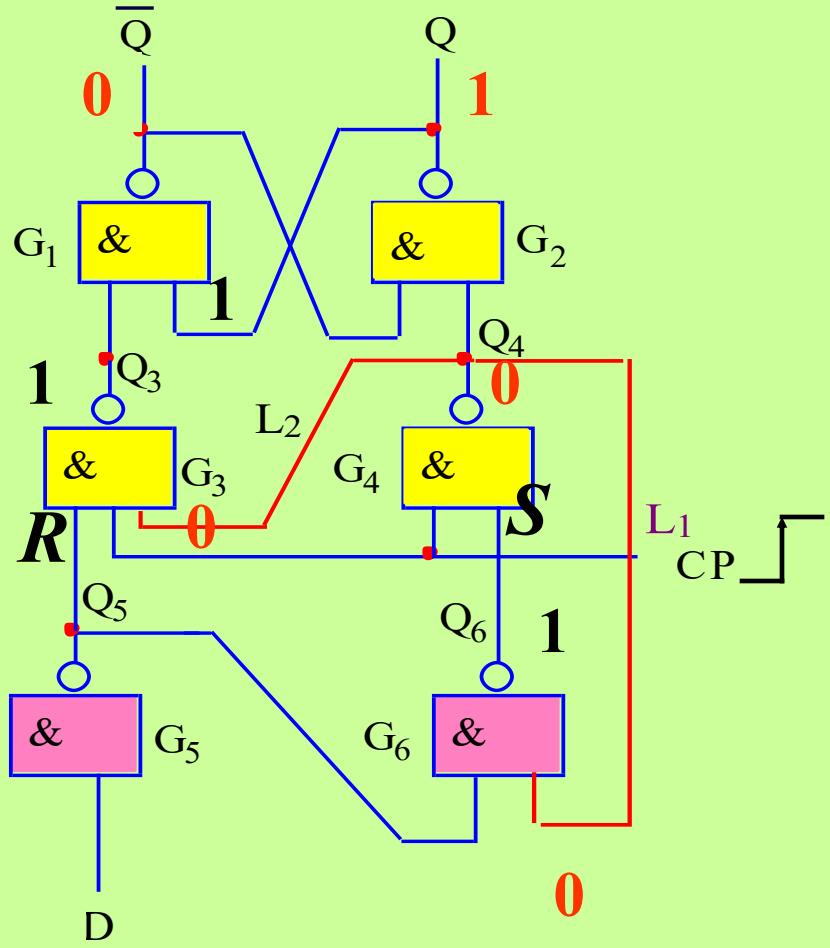




lilinear@zju.edu.cn

维持—阻塞边沿D触发器

为了克服空翻，在原电路的基础上引入三根反馈线。



①置1功能。

设：D=1

L_1称为置1维持线。

L_2称为置0阻塞线。

lilinear@zju.edu.cn

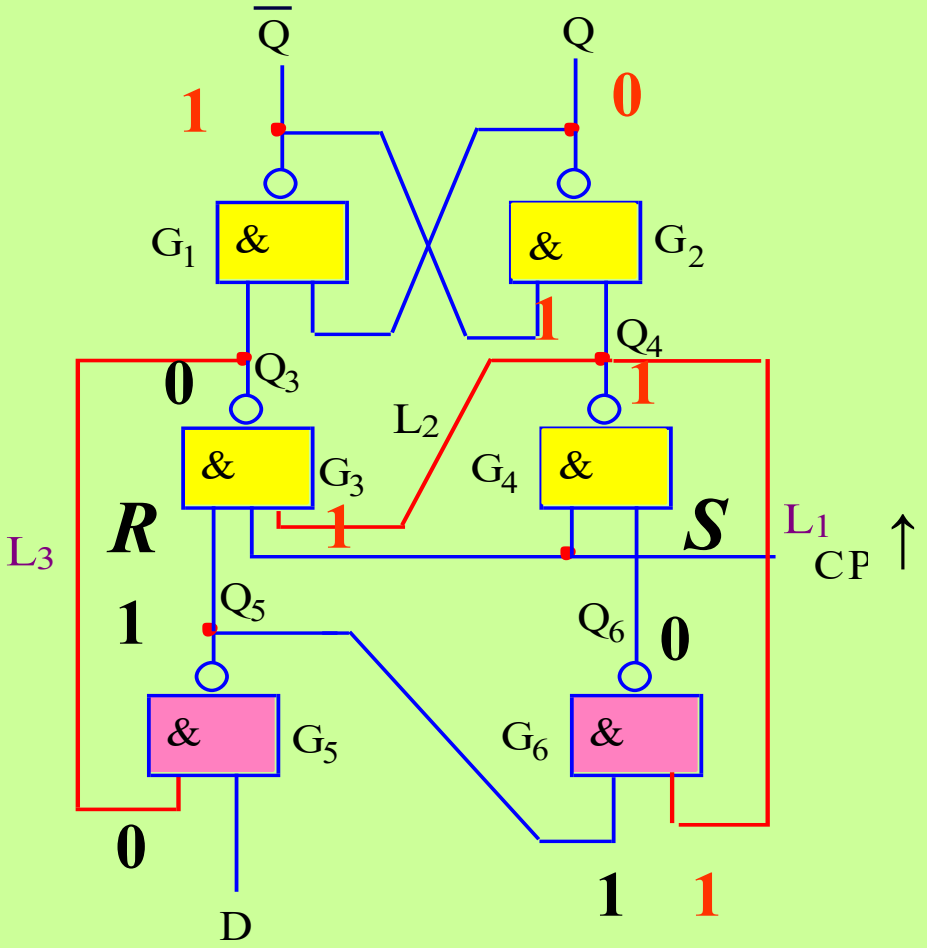
(2) 维持—阻塞边沿D触发器

②置0功能。

设：D=0

L_3称为置0维持线。

可见，引入了维持线和阻塞线后，将触发器的触发翻转控制在*CP*上跳沿到来的一瞬间，并接收*CP*上跳沿到来前一瞬间的*D*信号。



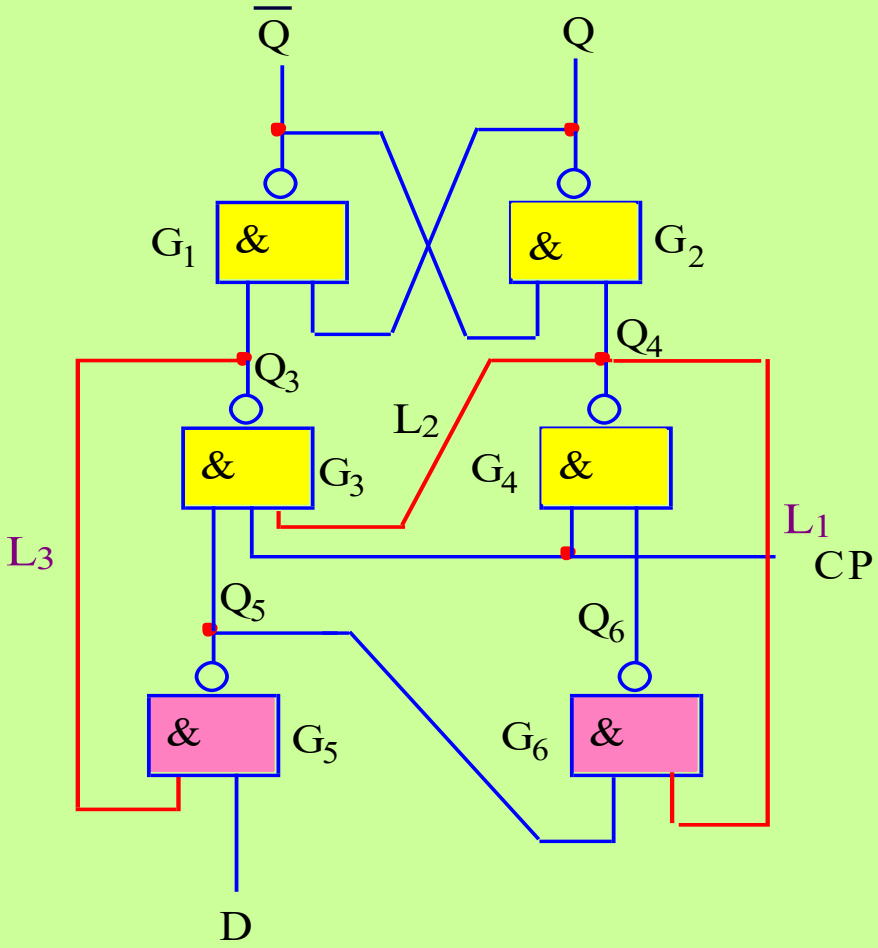
lilinear@zju.edu.cn

（2）维持—阻塞边沿D触发器

L_1称为置1维持线。

L_2称为置0阻塞线。

L_3称为置0维持线。



lilinear@zju.edu.cn

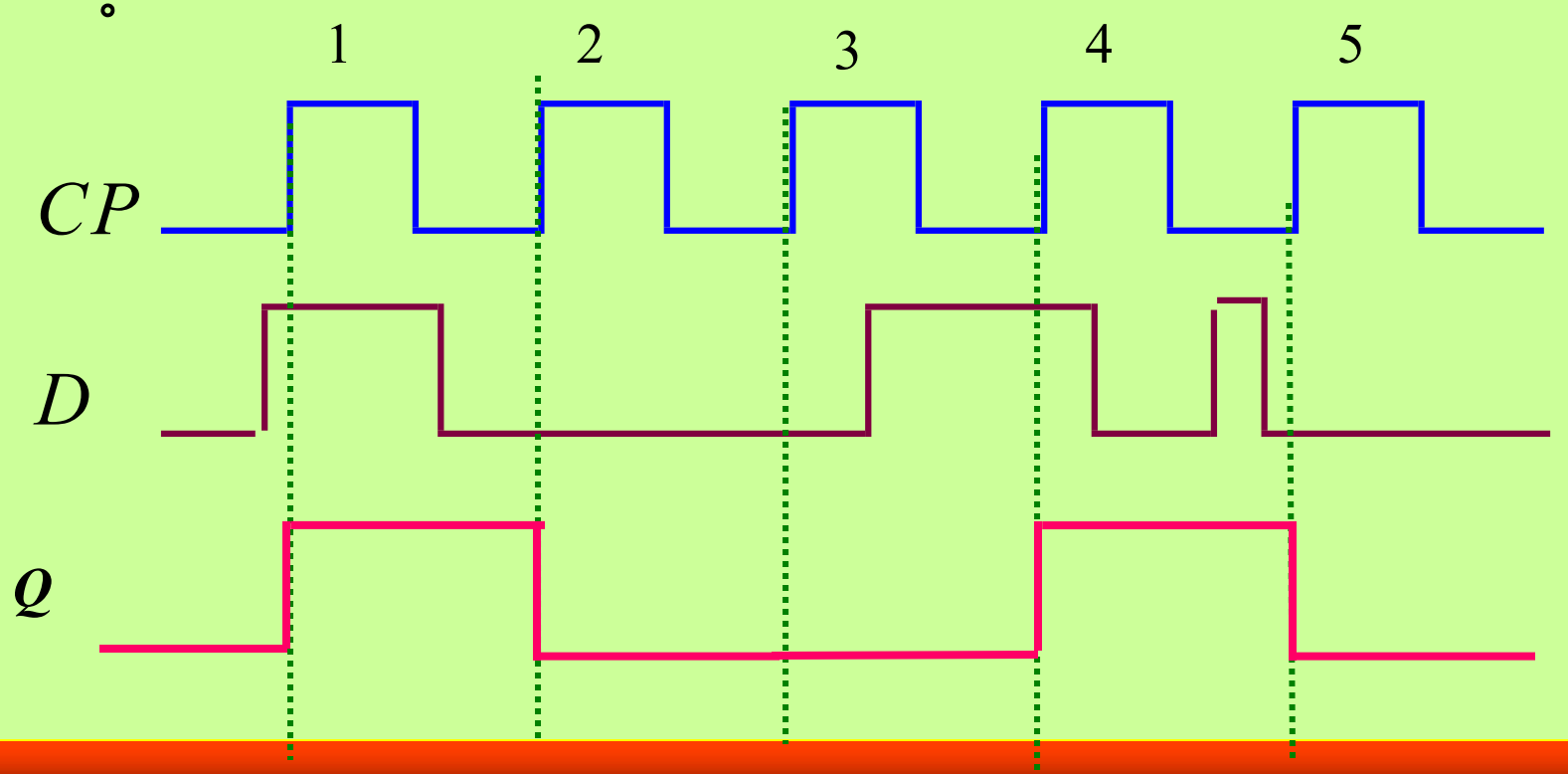
已知维持—阻塞D触发器的输入波形，

画出输出波形图。

解：在波形图时，应注意以下两点：

（1）触发器的触发翻转发生在CP的上升沿。

（2）判断触发器次态的依据是CP上升沿前一瞬间输入端D的状态。



lilinear@zju.edu.cn

D触发器逻辑功能表

1、$CP=0$ 时，Q保持不变

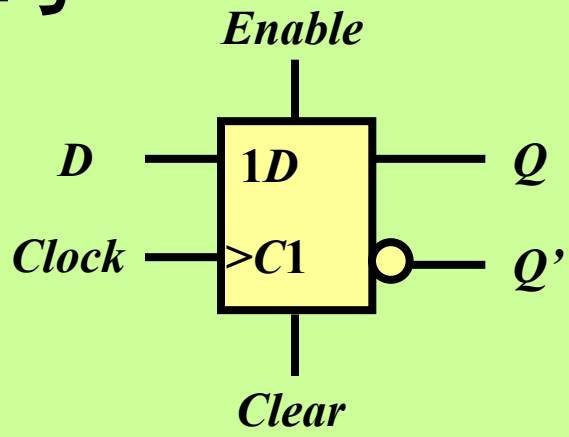
2、CP 上升沿时，Q状态按D变化

3、$CP=1$时，Q状态保持

结论： $Q^* = D$ **:CP上升沿有效**

lilinear@zju.edu.cn

边沿型D触发器的V代码




```
module D_flipflop (
  input D,                    // signals
  input Clock,
  input Enable,
  input Clear,
  output reg Q
);

  // execute on rising clock edge or Clear
  always @(posedge Clock or posedge Clear) begin
    if (Clear) begin
      Q <= 1'b0;      // assign 0 to Q on clear
    end else if (Enable) begin
      Q <= D;         // assign value from D to Q only if Enable is asserted
    end
  end

endmodule
```


lilinear@zju.edu.cn